Assignment-04

Question 1

What is a Defect? Explain the importance of defect tracking in software testing.

Answer

A defect (also called a bug or fault) is a flaw in a software work product, such as a requirement, design or code, which causes the software to behave in a way that differs from the expected behaviour. When the defective code is executed, it may produce a failure, that is, the software does not do what the user expects. The terms are related: a person makes an **error** (mistake), the error produces a **defect** in the product, and the defect may cause a **failure** during execution. For example, if a developer writes "if (age > 18)" where the requirement says age 18 and above is allowed, the user of age 18 will be wrongly rejected.

Importance of defect tracking

Defect tracking is the process of recording every defect from the time it is found until it is closed, and monitoring its status. It is important for the following reasons:

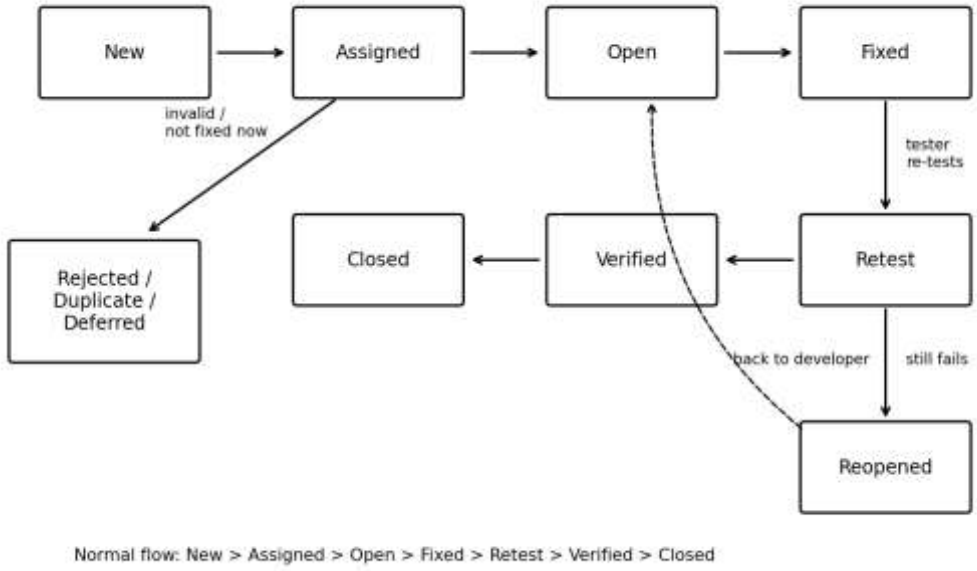
- **No defect is lost:** every defect is recorded with its details, so none is forgotten or ignored.
- **Clear ownership and status:** the team always knows who is responsible for a defect and at which stage it is.
- **Better communication:** testers, developers and managers refer to the same record, which reduces misunderstandings.
- **Prioritisation:** severe and urgent defects are fixed first, so that the resources are used properly.
- **Measuring quality:** the data (number, severity, module-wise distribution) shows the quality of the product and helps to decide the release.
- **Process improvement:** analysis of the defect causes shows the weak areas of development and testing, and helps to prevent similar defects.
- **Verification of fixes:** it ensures that every fixed defect is re-tested before closing.
- **History and audit:** a record is kept for future projects, regression testing and audits.
- **Cost control:** early tracking and fixing reduces the cost of correction.

Question 2

Explain the Defect Life Cycle with a suitable diagram.

Answer

The Defect Life Cycle (Bug Life Cycle) is the series of states through which a defect passes from its discovery up to its closure. It defines a common way of handling defects so that everyone in the project understands the status of a defect. The exact states can differ from one organisation or tool to another.



Explanation of the flow:

- The tester finds a defect and logs it in the tracking tool. Its state is **New**.
- The test lead or the project manager reviews it and gives it to a developer. The state is **Assigned**.
- The developer starts analysing and working on it: **Open**. If the developer decides that it is not a valid defect (rejected), a repeat of an existing defect (duplicate), or it is to be corrected in a later release (deferred), the defect moves out of the normal flow to the corresponding state.
- After correcting the code, the developer marks it **Fixed**.
- The tester re-tests the fix in the new build (**Retest**). If the defect is gone, the state becomes **Verified** and then **Closed**.
- If the defect still occurs, the tester marks it **Reopened** and it goes back to the developer; the cycle is repeated until the defect is properly fixed.

Question 3

What are the different states of a defect in the Defect Life Cycle? Explain each state.

Answer

The states of a defect describe the stage of its handling. The commonly used states are:

- **New:** the defect has been found and reported by a tester for the first time, and it is not yet reviewed.
- **Assigned:** the defect has been accepted for fixing and allotted to a developer or a development team by the test lead or the manager.
- **Open:** the developer has started to analyse the defect and work on its fix.
- **Fixed:** the developer has corrected the code and unit-tested the fix; the fix is ready to be given to the testers in the next build.
- **Pending Retest:** the fixed code has been delivered and is waiting for the tester to check it.
- **Retest:** the tester is re-executing the failed test case to confirm that the defect has been removed.
- **Verified:** the retest passed and the defect is confirmed as fixed (also regression is checked).

- **Closed:** the defect is finally resolved and no further action is needed.
- **Reopened:** the defect still exists after the fix, or has appeared again; it goes back to the developer.
- **Rejected:** the developer or lead decides that it is not a defect because the software is working as per the requirement.
- **Duplicate:** the same defect has already been reported earlier, so this report is linked to the original and closed.
- **Deferred (Postponed):** a valid defect whose fixing is planned for a future release because it has low priority or low impact, or time is short.
- **Cannot Reproduce / Need More Information:** the developer is not able to see the defect with the given steps and asks the tester for further details.

Question 4

Explain the process of Defect Tracking and Reporting in software testing.

Answer

Defect tracking and reporting is the systematic procedure of finding, documenting, communicating, monitoring and resolving defects with the help of a defect tracking tool. It is the central communication channel between testing and development. The process consists of the following steps:

- **1. Detect the defect:** during test execution the actual result does not match the expected result.
- **2. Confirm and analyse:** the tester repeats the test to confirm that it is reproducible, checks that it is not a duplicate and identifies the exact steps and conditions.
- **3. Report the defect:** a defect report is written with a clear title, steps to reproduce, expected and actual results, environment, severity, priority and attachments such as screenshots.
- **4. Review and assign:** the lead or a defect triage meeting verifies the defect, fixes its severity and priority and assigns it to a developer.
- **5. Fix:** the developer reproduces the defect, finds the root cause, corrects it, and updates the status.
- **6. Retest and regression:** the tester checks the fix and also checks that the fix has not broken other parts.
- **7. Close or reopen:** if it is fixed, the defect is closed; otherwise it is reopened with comments.
- **8. Track and monitor:** the status of all defects is watched continuously, with daily or weekly reports on the open, fixed and closed defects.
- **9. Report and analyse:** summary reports and metrics (defect density, defect trend, ageing) are made for the management, and the causes are analysed to improve the process.

Question 5

What information should be included in a Defect Report? Explain.

Answer

A defect report is the written description of a defect. It should be complete, clear and objective so that a developer can understand and reproduce the defect without further questions. It should include:

- **Defect ID:** a unique number generated by the tool.
- **Title / summary:** a short one-line statement of the problem.
- **Description:** a detailed explanation of what is wrong.
- **Steps to reproduce:** numbered steps with the data used, starting from a known state.

- **Expected result:** the correct behaviour according to the requirement.
- **Actual result:** the behaviour that was observed.
- **Environment details:** operating system, browser or device, build or version, and configuration.
- **Module or component:** the part of the application where the defect occurs.
- **Severity and priority:** the impact and the urgency.
- **Status:** the present state in the life cycle.
- **Reported by, assigned to and dates:** for responsibility and ageing.
- **Attachments:** screenshots, videos and logs.
- **Reproducibility:** always, intermittent or once.
- **Reference:** the related test case ID or requirement ID.
- **Comments and history:** discussions, changes of status and the resolution details.

A good report describes only one defect, uses simple and neutral words, does not blame anyone and gives enough evidence.

Question 6

Differentiate between Defect Severity and Defect Priority with suitable examples.

Answer

Basis	Defect Severity	Defect Priority
Meaning	The degree of impact of the defect on the functioning of the software.	The order or urgency in which the defect should be fixed.
Concerned with	Technical impact on the system.	Business need and schedule.
Decided by	Normally the tester or QA, who judges the technical effect.	Normally the project manager, product owner or the lead, with the customer's view.
Based on	Seriousness of failure (crash, data loss, wrong output).	Importance to the business, deadlines, visibility to the customer.
Levels	Critical, Major, Minor, Trivial.	High, Medium, Low (or P1, P2, P3).
Change over time	Usually remains fixed, unless the nature of the defect is understood differently.	Can change with the release plan and business situation.
Focus	How bad is the defect?	How soon should it be fixed?

Examples of the four combinations

- **High severity and high priority:** the application crashes when the user clicks Pay Now on an e-commerce site. The impact is critical and it must be fixed immediately.
- **High severity and low priority:** the application crashes in a rarely used feature, for example a report which only one administrator opens once in a year. The impact is serious but the urgency is low.
- **Low severity and high priority:** the company name is misspelt on the home page just before a customer demonstration. The function is not affected, but it is very visible, so it must be fixed at once.
- **Low severity and low priority:** a small alignment problem in the help page. It can be fixed in a later release.

Question 7

Explain the Defect Management Process in detail.

Answer

Defect management is the complete process of identifying, recording, classifying, resolving and preventing defects throughout the software development life cycle. Its aim is to find the defects early, fix them at the lowest cost, and use the information to improve the process and to prevent defects in the future. The main stages of the process are:

- **1. Defect prevention:** the activities that reduce the chance of defects, such as reviews of requirements and design, coding standards, training and use of checklists. Prevention is cheaper than correction.
- **2. Defect discovery:** defects are found by static testing (reviews, inspections) and by dynamic testing (executing test cases). The defect is acknowledged after it is confirmed.
- **3. Defect reporting and recording:** the defect is documented in the tracking tool with all required details.
- **4. Defect classification and prioritisation:** the defect is classified by type, severity, priority and module, usually in a defect triage meeting.
- **5. Defect resolution:** the developer reproduces the problem, finds the root cause, fixes the defect and updates the status, and the fix is included in a build.
- **6. Verification and closure:** the tester performs retesting and regression testing; the defect is closed only after successful verification.
- **7. Defect analysis and reporting:** the management is informed with defect reports and metrics, such as defect density and trends; the root cause analysis is performed.
- **8. Process improvement:** the lessons learnt are used to improve the development and testing process and to prevent repeating the same defects.

Question 8

What are the main activities involved in Defect Management? Explain each activity.

Answer

- **Defect prevention:** reduce the probability of defects by methods like inspections, risk analysis, standards and training of the team. Example: a requirement review that removes ambiguity.
- **Defect identification (detection):** find the defects through testing and reviews. A deviation from the expected result is noted as a defect.
- **Defect recording and reporting:** document the defect in a tracking tool with the required information. Proper recording makes it possible to fix and track it.
- **Defect classification and prioritisation:** assign the type (functional, UI, performance, security), severity and priority so that the more important defects get attention first.
- **Defect assignment:** hand over the defect to the right developer or team and set a target date.
- **Defect resolution (fixing):** the developer analyses the root cause, corrects the code and performs unit testing of the fix.
- **Defect verification (retesting and regression):** the tester repeats the failed test and also related tests to ensure that the fix works and has not created new defects.
- **Defect closure:** after verification the defect is closed with proper comments.
- **Defect tracking and monitoring:** follow the status of all defects, escalate those which are delayed and review them in regular meetings.
- **Defect analysis (root cause analysis) and reporting:** study the data and the causes, create metrics and reports for the stakeholders.
- **Process improvement:** use the findings for preventive and corrective action in the process.

Question 9

Explain the role of Defect Management Tools in software testing.

Answer

Defect management tools (bug tracking tools) are software applications that help the team to record, assign, track, report and analyse defects. Examples are Jira, Bugzilla, Mantis, Redmine, Azure DevOps and Trello-based workflows. Their role is as follows:

- **Central repository:** all defects are stored in one place which is accessible to all the team members, even if they are at different locations.
- **Standard recording:** templates guarantee that the details like steps, environment and attachments are always entered.
- **Workflow management:** the defect moves through the life cycle states (new, assigned, fixed, closed) according to defined rules and permissions.
- **Assignment and notification:** the defects are assigned to the responsible persons, and e-mail or messages alert them about a change of status.
- **Tracking and monitoring:** the current status, age and owner of every defect can be seen at any moment.
- **Search and filtering:** defects can be found by module, severity, status or date, and duplicate defects can be detected.
- **Reports, dashboards and metrics:** the tool produces charts and reports such as defects by severity, trend of open and closed defects, and defect density.
- **Traceability and integration:** links defects to test cases, requirements and builds, and integrates with test management, version control and CI tools.
- **History and audit trail:** records every change, comment and decision.
- **Better teamwork and efficiency:** reduces manual work and misunderstandings between testers and developers, and saves time and cost.

Question 10

What are the important features of a good Defect Tracking Tool? Explain.

Answer

- **Easy to use interface:** simple forms and navigation so that a defect can be logged quickly with little training.
- **Customisable fields and workflow:** the ability to add fields (severity, priority, module, build) and change the life cycle states according to the project.
- **Role-based access control:** different permissions for testers, developers, managers and customers to protect data.
- **Attachment support:** screenshots, videos and log files can be attached to the defect.
- **Notifications and alerts:** automatic e-mail or in-tool messages when a defect is assigned or its status changes.
- **Search, sort and filter:** powerful queries and saved filters, and duplicate defect detection.
- **Reports and dashboards:** ready-made and custom reports, charts and metrics.
- **Integration:** with test management tools, requirement tools, version control, build tools and chat applications.
- **Traceability:** links among requirements, test cases and defects.
- **History and audit trail:** a full record of changes, comments and who made them.
- **Collaboration:** comments, mentions and discussion threads.

- **Scalability, performance and security:** can handle many projects, users and defects, with backup and secure access.
- **Multi-project and multi-platform support:** web and mobile access, support for multiple projects and versions.
- **Cost and support:** reasonable licence cost, good documentation and technical support.

Question 11

Explain how defects are identified, recorded, assigned, fixed, retested, and closed during the Defect Management Process.

Answer

- **Identified:** while executing a test case the tester compares the actual result with the expected result. If they differ, the tester repeats the test, checks the test data and the environment, and confirms that it is a genuine and reproducible defect (defects are also found in reviews and by users). Example: on entering a valid username and password the login page shows an error.
- **Recorded:** the tester first checks the tool for a duplicate, then logs the defect with a title, steps to reproduce, expected and actual results, environment, severity, attachments and the linked test case. The initial status is New.
- **Assigned:** the test lead or the triage team reviews the defect, confirms whether it is valid, decides the severity and priority, and assigns it to the responsible developer with a target date. The status becomes Assigned.
- **Fixed:** the developer reproduces the defect, finds the root cause, changes the code, performs unit testing and checks in the change. The status changes to Open while working, and then Fixed, and the fix is delivered in a new build with a note on what was changed.
- **Retested:** the tester executes the same test case again on the new build with the same data. Regression tests around the changed area are also executed. If the defect is no longer present and no new problem has been created, the status is Verified. If the defect still exists, the tester marks it Reopened with comments and the cycle returns to the developer.
- **Closed:** after successful verification, the defect is closed with final remarks. The closed defect remains in the database for history, metrics and future reference.

This sequence is repeated for every defect and continues until all the defects meeting the release criteria are closed.

Question 12

What are Test Metrics? Explain their importance in software testing.

Answer

A metric is a quantitative measure of a characteristic of a product or a process. Test metrics are measures that are collected and analysed to understand the status, quality, progress and efficiency of testing. For example: number of test cases executed, percentage of tests passed, number of defects found, defect density, and requirement coverage. By themselves, raw numbers have little meaning; they become useful when they are interpreted and compared with the targets.

Importance of test metrics

- **Progress monitoring:** they show how much of the planned testing is complete and whether the project is on schedule.
- **Quality assessment of the product:** the numbers of defects, their severity and trend show how stable and reliable the software is.

- **Release decision:** the management can decide whether the exit criteria are fulfilled and whether the product is ready to go live.
- **Measuring test effectiveness and efficiency:** show how well the testers find defects and how much effort is used.
- **Estimation and planning:** historical metrics help to estimate effort, time and cost for future projects.
- **Process improvement:** show the weak modules and the stages where defects escape, which helps to improve the process.
- **Risk identification:** poor coverage or increasing open defects warn the team early.
- **Objective communication:** give facts to the stakeholders in the form of numbers and graphs rather than opinions.
- **Cost control:** help to measure the cost of quality and avoid wasted effort.

Question 13

Explain different types of Test Metrics used to measure testing activities.

Answer

Test metrics can be divided into the following types:

1. Process metrics

Measure the performance of the testing process. Examples: defect detection efficiency, test case effectiveness, review efficiency, and test cycle time.

2. Product metrics

Measure the quality of the software product. Examples: defect density, defects by severity, mean time between failures (MTBF) and number of open critical defects.

3. Project metrics

Measure the progress, cost and resources of the testing project. Examples: effort variance, schedule variance, cost of testing and productivity.

4. Commonly used metrics in practice

- **Test execution metrics:** number of test cases planned, executed, passed, failed, blocked and not run. Execution percentage = executed / total x 100.
- **Test coverage metrics:** requirement coverage (requirements covered by test cases / total requirements x 100), code coverage (statement, branch, path) and test coverage of features.
- **Defect metrics:** total defects, open and closed defects, defects by severity, priority, module and type; defect density = defects / size (KLOC); defect age; defect leakage = defects found after release / total defects x 100; defect removal efficiency.
- **Effort and cost metrics:** planned vs actual effort, cost of testing, cost per defect.
- **Schedule metrics:** planned vs actual dates of milestones; schedule variance.
- **Productivity metrics:** test cases designed or executed per person per day.
- **Automation metrics:** percentage of tests automated, automation execution time saved, and script failure rate.
- **Test effectiveness metrics:** defect detection percentage (DDP) = defects found in testing / (defects found in testing + defects found after release) x 100.

Metrics may also be classified as **base metrics** (raw data collected directly, such as number of test cases) and **derived metrics** (calculated from base metrics, such as percentage of passed tests).

Question 14

What is Test Reporting? Explain the importance of test reports in software testing.

Answer

Test reporting is the activity of collecting the information about the testing performed, analysing it and communicating the results to the stakeholders in a documented form. A test report is the document that summarises the test activities, the results, defects, the status of the testing against the plan, and the conclusion or recommendation about the quality of the product. Reports are prepared during testing (progress or status report) and at the end of testing (test summary report).

Importance of test reports

- **Communicate status:** the managers and customers get a clear picture of what has been tested and what is pending.
- **Support decision making:** the report helps to decide whether the software is ready for release or more testing is needed.
- **Show product quality:** the defect data and coverage show the quality and the remaining risk.
- **Accountability and transparency:** record the work done by the test team and show if the plan was followed.
- **Identify problems early:** delays, blocked tests and high defect counts are noticed in time.
- **Evidence for audit and compliance:** a permanent record of testing for contracts, standards and audits.
- **Basis for improvement:** lessons learnt and metrics help to improve future projects.
- **Customer confidence:** a clear report builds trust with the client.

Question 15

What are the main contents of a Test Report? Explain.

Answer

A test report (test summary report, as per IEEE 829) usually contains the following:

- **Report identifier and project details:** name of the project, version or build, date, author and the period of testing.
- **Introduction and objective:** purpose of the testing and of the report.
- **Test summary:** short overview of what was tested and the overall result.
- **Scope of testing:** features and test levels covered, and also what was not tested, with reasons.
- **Test environment:** hardware, software, browsers, devices, tools and test data used.
- **Test execution results:** number of test cases planned, executed, passed, failed, blocked and not executed, with percentages.
- **Defect summary:** total defects found, open and closed, by severity, priority and module, and details of important unresolved defects.
- **Coverage:** requirement and code coverage achieved.
- **Deviations from the plan:** differences from the test plan, such as skipped tests or schedule change, and the reasons.
- **Metrics and charts:** graphs of test progress, defect trend and defect distribution.

- **Risks and issues:** remaining risks, known limitations and workarounds.
- **Exit criteria evaluation:** whether the planned completion criteria were met.
- **Conclusion and recommendations:** the overall assessment of quality and whether the software is recommended for release (Go / No-Go).
- **Lessons learnt:** what went well and what should be improved.
- **Approvals:** names and signatures of the reviewers.

Question 16

Explain different Test Reporting Techniques used in software testing.

Answer

Test reporting techniques are the different ways in which the test information is presented to the stakeholders. The technique is chosen according to the audience and the purpose.

- **Test summary report:** a formal document prepared at the end of the test cycle with the results, defects, coverage and recommendation.
- **Daily or weekly status report:** a short regular report on the progress, the defects found, the blocked items and the plan for the next period. It is used in project meetings.
- **Defect report and defect summary report:** details of individual defects and a statistical summary of the defects by status, severity and module.
- **Dashboards:** a single visual page, often updated automatically by tools such as Jira or Azure DevOps, which shows the live status of testing.
- **Graphs and charts:** bar charts, pie charts, trend lines, burn-down charts and defect trend charts to show data in an easy way. Example: the number of open defects per day.
- **Tables and metrics reports:** structured tables of the numbers of test cases and defects, the coverage and the metrics.
- **Traceability matrix reports:** show the mapping between requirements, test cases and defects, and thereby the coverage.
- **Test execution report from tools:** the automatic reports generated by automation or test management tools (for example TestNG or Selenium reports).
- **Verbal or presentation reporting:** daily stand-up, review meetings, defect triage meetings and demos for the management and the client.
- **E-mail and instant messages:** for quick alerts, such as a blocking defect or a build failure.
- **Exception or escalation reporting:** a special report of critical issues and risks that need immediate attention of the management.

Question 17

Differentiate between Test Metrics and Test Reports.

Answer

Basis	Test Metrics	Test Reports
Meaning	Quantitative measurements of testing activities and product quality.	A document or presentation that communicates the testing status and results.
Nature	Data and numbers (for example 90% execution, 25 defects).	Narrative plus data, charts and conclusions.
Purpose	To measure progress, quality, effectiveness and efficiency.	To inform the stakeholders and to support decisions.

Basis	Test Metrics	Test Reports
Form	Values, formulas and ratios.	Formal or informal document, dashboard or presentation.
Relationship	They are the input and form a part of the report.	They use the metrics, and add interpretation and recommendation.
Timing	Collected continuously during testing.	Produced periodically (daily or weekly) and at the end of testing.
Audience	Mostly used by the test team and the managers for analysis.	Meant for managers, developers, customers and other stakeholders.
Interpretation	Needs analysis to give meaning.	Already contains the analysis, conclusions and suggestions.
Example	Defect density = 2 defects per KLOC.	Test Summary Report of Release 2.0 stating that the build is ready for release.

Question 18

Explain how defect metrics can be used to measure the quality of software.

Answer

Defect metrics are the measures obtained from the defects found in the software. The number, the nature and the trend of the defects are a direct indication of the quality of the product, as a product with many serious defects is considered to be of poor quality. The main defect metrics and their use are:

- **Defect density:** number of defects / size of the software (per KLOC or per function point). A high density in a module shows that the module is of poor quality and risky. It also allows comparison of the modules and releases.
- **Defect distribution by severity:** the number of critical, major and minor defects. A large number of critical defects means that the product is not stable enough for release.
- **Defect distribution by module (defect clustering):** shows the modules which contain most of the defects, so that more testing or redesign can be planned there.
- **Defect trend (open vs closed over time):** if the number of new defects found per day decreases and the number of closed defects increases, the quality is improving and the product is approaching the release.
- **Defect arrival and fix rate:** a high arrival rate late in the project is a warning sign.
- **Defect leakage (escape rate):** defects found after release / total defects x 100. A high value indicates poor quality of the product and of the testing.
- **Defect removal efficiency (DRE):** defects removed before release / (defects removed before release + defects found after release) x 100. A value close to 100% indicates good quality control.
- **Defect age and reopen rate:** long ageing and many reopened defects indicate poor fixes and unstable code.
- **Defects by type or cause:** requirement, design or coding defects show where the quality problem starts.
- **Mean time between failures (MTBF):** a high MTBF means that the software is reliable.

By comparing these metrics with the targets and with earlier releases, the management can judge the quality objectively and decide on the release.

Question 19

Explain how Test Metrics can be used to measure the effectiveness and efficiency of a testing team.

Answer

Effectiveness means how well the testing team achieves its objective of finding defects and giving the right information (doing the right thing). **Efficiency** means how well the team uses time, effort and cost to do so (doing the thing right). Metrics help to measure both.

Measuring effectiveness

- **Defect detection percentage (DDP):** defects found by the test team / (defects found by the test team + defects found after release) x 100. A high DDP shows an effective team.
- **Defect leakage:** a low leakage to production shows effective testing.
- **Test case effectiveness:** number of defects found by test cases / number of test cases executed x 100.
- **Requirement and code coverage:** a high coverage shows that the testing is complete.
- **Valid defect ratio and rejected defects:** the ratio of accepted defects to total reported shows the quality of the reporting.
- **Defects found per severity:** finding the more severe defects early indicates effective testing.

Measuring efficiency

- **Productivity:** test cases designed or executed per person per day.
- **Defects found per hour or per person-day of testing.**
- **Effort variance:** (actual effort - planned effort) / planned effort x 100.
- **Schedule variance:** the difference between the planned and actual dates.
- **Cost per defect found or per test case executed.**
- **Automation percentage and time saved** by automation.
- **Test cycle time and rework:** time taken to complete one test cycle and the number of tests repeated because of errors.

These metrics should be used to guide and improve the team and not to punish individuals; they should be used together, because for example a high number of test cases executed with low defect detection may not mean good testing.

Question 20

How can metrics be used to identify problems and improve testing effectiveness? Explain.

Answer

Metrics make the problems visible by comparing the actual values with the planned values and with earlier projects. When the cause is analysed, corrective and preventive actions can be taken. Examples:

Identifying problems

- **Low execution percentage or many blocked tests** show problems with the builds, the environment or the schedule.
- **High defect density in a module** shows poor design or coding in that module and a need for more testing or code review.
- **Defect leakage to production** shows that the test cases or the test environment are not adequate.

- **Low requirement coverage** shows missing test cases, so some requirements are untested.
- **Many reopened defects** show poor quality of the fixes or poor retesting.
- **Large number of rejected or duplicate defects** show that the testers do not understand the requirements or the reporting procedure is weak.
- **Increasing defect arrival rate near the release** warns that the product is unstable.
- **High effort or schedule variance** shows wrong estimation or inefficiency.
- **Long defect ageing** shows a delay in the fixing process or a lack of developer resources.

Improving effectiveness

- **Add or improve test cases** in the weak areas, using better design techniques (EP, BVA, decision tables).
- **Focus on risky modules:** put more effort in the modules with a high defect density (defect clustering).
- **Root cause analysis:** find why the defects occurred (unclear requirements, coding mistakes) and take preventive action, such as reviews and training.
- **Improve the process:** better entry and exit criteria, stronger smoke testing before accepting a build, and stable test environments.
- **Strengthen regression and automation** where repeated work is large.
- **Improve the defect reports** with better templates and training to reduce rejections and duplicates.
- **Revise estimates and plans** using actual productivity and variance data.
- **Set targets and review metrics regularly** (weekly meetings), and compare with the previous releases for continuous improvement.

In this way, the cycle of **measure, analyse, improve and measure again** makes the testing more effective and efficient from project to project.